

Поток управления в NASM

Повторение. Язык Ассемблера и регистры

- Как процессор понимает, какую инструкцию сейчас выполнять?

Повторение. Язык Ассемблера и регистры

- Как процессор понимает, какую инструкцию сейчас выполнять?
- Кажется, все очень просто: мы сообщаем процессору первую инструкцию программы, а дальше он выполняет программу последовательно.

Повторение. Язык Ассемблера и регистры

- Как процессор понимает, какую инструкцию сейчас выполнять?
- Кажется, все очень просто: мы сообщаем процессору первую инструкцию программы, а дальше он выполняет программу последовательно.
- Но всегда ли процессор выполняет программу последовательно: инструкцию за инструкцией, сверху вниз смотря по исходному коду?

Повторение. Язык Ассемблера и регистры

- Как процессор понимает, какую инструкцию сейчас выполнять?
- Кажется, все очень просто: мы сообщаем процессору первую инструкцию программы, а дальше он выполняет программу последовательно.
- Но всегда ли процессор выполняет программу последовательно: инструкцию за инструкцией, сверху вниз смотря по исходному коду?
- Конечно нет! Как минимум, у нас есть функции. Вызвав функцию, мы “отправляем” процессор исполнять другой кусок кода.

Повторение. Язык Ассемблера и регистры

- Вернемся к вопросу. Как мы указываем процессору, какую инструкцию исполнять?

Повторение. Язык Ассемблера и регистры

- Вернемся к вопросу. Как мы указываем процессору, какую инструкцию исполнять?
- Давайте “пронумеруем” инструкции. Номер инструкции, которую нужно исполнить будем хранить в регистре EIP (Instruction Pointer).

Поток управления. Регистр EIP

Рассмотрим пример, как EIP однозначно связан с текущей исполняемой инструкцией. Для этого:

1. Скомпилируем ассемблерный код в исполняемый файл
2. Дизассемблируем (обратный процесс) обратно в ассемблерный код
3. Запустим под дебаггером исполняемый код и посмотрим на значение EIP, сравним его с адресами в дизассемблированном коде

Способы передачи управления

1. **Безусловные – происходят всегда.**

Всегда прыгаем по переданному адресу

- а. Вызов функции
- б. Безусловный прыжок

2. **Условные – происходят только при выполнении условия.**

Если условие выполняется – прыгаем, если нет – исполняем следующую инструкцию

- а. много видов вариаций условного прыжка

Безусловные переходы

Безусловные – происходят всегда.

Всегда прыгаем по переданному адресу

- a. Вызов функции
- b. Безусловный прыжок

Безусловный переход – просто запись в EIP нового адреса инструкции. В каком-то смысле, это аналог **mov EIP, <address>**. Напрямую в EIP нельзя класть значение в целях безопасности

Безусловные переходы

Безусловные – происходят всегда.

Всегда прыгаем по переданному адресу

- a. Вызов функции
- b. Безусловный прыжок

Безусловный переход – просто запись в EIP нового адреса инструкции. В каком-то смысле, это аналог **mov EIP, <address>**. Напрямую в EIP нельзя класть значение в целях безопасности

Мы уже поняли, что достаточно записать в регистр EIP адрес нужной инструкции и процессор сразу же начнет исполнять ее. Но как же нам узнать адрес во время написания кода на ассемблере?

Безусловные переходы

Безусловные – происходят всегда.

Всегда прыгаем по переданному адресу

- a. Вызов функции
- b. Безусловный прыжок

Безусловный переход – просто запись в EIP нового адреса инструкции. В каком-то смысле, это аналог **mov EIP, <address>**. Напрямую в EIP нельзя класть значение в целях безопасности

Мы уже поняли, что достаточно записать в регистр EIP адрес нужной инструкции и процессор сразу же начнет исполнять ее. Но как же нам узнать адрес во время написания кода на ассемблере?

Тут на помощь приходят метки

Безусловные переходы. Метки

Метка – инструкция программе-ассемблеру, что во время ассемблирования надо запомнить адрес текущего места в коде.

Метки пропадают после компиляции – они заменяются на адреса. Таким образом, программист может “помечать” участки кода, а ассемблер заменит эти метки адресами

```
main:  
    mov eax, -2  
    call run_case
```

```
0804922e <main>:  
804922e: b8 fe ff ff ff      mov     eax,0xffffffff  
8049233: e8 d5 ff ff ff      call    804920d <run_case>
```

Безусловные переходы. Метки

```
main:  
    mov eax, -2  
    call run_case
```

Исходный код ассемблера. Вызов по метке “run_case”

```
0804922e <main>:  
804922e: b8 fe ff ff ff      mov     eax,0xffffffff  
8049233: e8 d5 ff ff ff      call   804920d <run_case>
```

Дизассемблированный код.

Метка “run_case” заменилась на адрес 0x804920d

Безусловные переходы. Метки.

Метки бывают глобальными и локальными.

Глобальные – видны во всей программе. До сих пор мы пользовались только ними

Локальные – видны только внутри последней глобальной метки. Локальные метки начинаются с точки.

Безусловные переходы. Метки.

локальные метки:

.is_positive
.is_zero
.is_negative
.finish

Пользоваться ими можно
только внутри метки
dispatch_sign.

Поэтому в разных функциях
можно объявлять одинаковые
локальные метки

```
2 dispatch_sign:
3     cmp eax, 0
4     je .is_zero
5     jl .is_negative
6
7     ; fallback into positive value
8
9 .is_positive:
10    mov eax, positive_msg
11    jmp .finish
12
13 .is_zero:
14    mov eax, zero_msg
15    jmp .finish
16
17 .is_negative:
18    mov eax, negative_msg
19    jmp .finish
20
21 .finish:
22    call print_msg
23    ret
```


Условные переходы

Условные переходы происходят только при выполнении условия.

Если условие выполняется – прыгаем, если нет – исполняем следующую инструкцию.

Условие перехода – это флаги ZF, CF, SF, OF и их комбинации.

Но неужели нам придется каждый раз вручную выставлять эти флаги для простого сравнения чисел?

Условные переходы. Сравнение

Для сравнения чисел существует команда

СМР <ОР1> <ОР2>

Эта команда:

1. вычитает **<ОР1> - <ОР2>**
2. выставляет флаги после вычитания
3. результат вычитания нигде не сохраняет

Условные переходы. Сравнение

Для сравнения чисел существует команда

CMP <OP1> <OP2>

Эта команда:

1. вычитает **<OP1> - <OP2>**
2. выставляет флаги после вычитания
3. результат вычитания нигде не сохраняет

Важное следствие (некоммутативность перестановки операндов):

CMP <OP1>, <OP2> не то же что **CMP <OP2>, <OP1>**

Условные переходы. Сравнение

Эта команда:

1. вычитает **<OP1>** - **<OP2>**
2. выставляет флаги после вычитания
3. результат вычитания нигде не сохраняет

Давайте разберем на примере

Условные переходы. Команды

Все прыжки имеют одинаковый синтаксис:

J <label>**

где **J**** – конкретная команда прыжка (их много)

<label> – метка куда прыгать в случае выполнения условия

Условные переходы. Равенство и неравенство

Условие зависит от типа перехода:

1. **JE** или **JZ** – jump if **zero** ($ZF == 0$)
2. **JNE** или **JNZ** – jump if **not zero** ($ZF != 0$)

Условные переходы. Беззнаковое сравнение

1. **JA** – jump if **above** ($CF == 0$ и $ZF == 0$)
2. **JAE** – jump if **above or equal** ($CF == 0$)
3. **JB** – jump if **below** ($CF == 1$)
4. **JBE** – jump if **below or equal** ($CF == 1$ или $ZF == 1$)

Условные переходы. Знаковое сравнение

1. **JG** – jump if **greater**
2. **JGE** – jump if **greater or equal**
3. **JL** – jump if **less**
4. **JLE** – jump if **less or equal**

Условные переходы. Все команды

КОП		Флаги	Описание
JO		OF = 1	overflow
JS		SF = 1	sign
JZ	JE	ZF = 1	zero/equal
JC	JB	CF = 1	below
	JBE	CF = 1 или ZF = 1	below or equal
JNC	JAE	CF = 0	above or equal
	JA	CF = 0 и ZF = 0	above
	JL	SF $\neq$ OF	less
	JLE	SF $\neq$ OF или ZF = 1	less or equal
	JGE	SF = OF	greater or equal
	JG	SF = OF и ZF = 0	greater
JECXZ		ECX = 0	

Условные переходы. Циклы

С помощью условных переходов можно сделать циклы

Чтоб завести цикл, мы должны:

1. Проверить условие.
2. Если условие выполнено – пойти внутрь цикла и продолжить работу
3. Если условие не выполнено – завершить цикл
4. В конце тела цикла прыгнуть на проверку условия

Условные переходы. Циклы

```
main:
    xor ebx, ebx           ; i = 0

.loop:
    cmp ebx, 5             ; while (i < 5)
    jge .done

    mov eax, ebx
    call io_print_hex
    call io_newline

    inc ebx
    jmp .loop

.done:
    xor eax, eax
    ret
```

Рекап

Сегодня узнали:

1. Про регистр EIP. Переход – изменение значения в этом регистре
2. Метки – способ пометить точку программы. Подменяются адресом во время компиляции
3. Метки бывают локальными. Они видны только внутри ближайшей сверху глобальной метки.
4. Безусловные переходы – происходят всегда. Просто меняется EIP

Рекап

5. Условные переходы – происходят по флагам. Если происходит – прыгаем по метке. Если нет – идем на следующую инструкцию
6. Для сравнения чисел нужно использовать CMP. Она вычитает первый операнд из второго. **CMP AX, DX** не то же что **CMP DX, AX**
7. С помощью локальных меток и условных переходов можно сделать циклы

Вопросы